

PROYECTO

Implementación de una Nube Personal con Nextcloud

Infraestructura, Contenedores y Automatización

Presentado por la Familia Escobar

Área: Computación y Programación

Abril 2026

1. Introducción

1.1 ¿Qué es una nube personal?

Una nube personal es un servicio de almacenamiento y sincronización de archivos que funciona en un servidor propio, en lugar de depender de servidores de terceros como Google, Microsoft o Apple. Permite guardar documentos, fotos, vídeos y cualquier tipo de archivo, accediendo a ellos desde cualquier dispositivo conectado a internet.

La principal ventaja de una nube personal es el control total sobre los datos: nadie más tiene acceso a la información almacenada, a diferencia de los servicios comerciales donde los archivos residen en servidores de empresas que pueden analizar, compartir o perder esos datos.

1.2 Objetivo del proyecto

Implementar un servicio de almacenamiento en la nube privado, seguro y accesible desde cualquier lugar del mundo, utilizando tecnologías de código abierto (open source) y un servidor dedicado de bajo consumo energético (NUC), con un costo operativo mínimo.

1.3 Comparación con servicios comerciales

Característica	Google Drive	Dropbox	Nube Personal (Nextcloud)
Almacenamiento	15 GB gratis / 2 TB \$10/mes	2 GB gratis / 2 TB \$12/mes	1.7 TB (disco propio)
Costo anual	\$120 USD (2 TB)	\$144 USD (2 TB)	~\$46 USD (dominio + luz)
Privacidad	Google accede a datos	Dropbox accede a datos	Control total
Usuarios	Limitado por plan	Limitado por plan	Ilimitados
Personalización	Mínima	Mínima	Total
Disponibilidad	99.9%	99.9%	Depende del servidor

2. Conceptos Fundamentales y Glosario Técnico

A continuación se explican los conceptos y términos técnicos utilizados a lo largo del proyecto, organizados por categoría.

2.1 Redes y Comunicaciones

- **IP (Internet Protocol):** Dirección numérica única que identifica a cada dispositivo en una red. Es como la dirección postal de una computadora. Ejemplo: 192.168.1.69.
- **DNS (Domain Name System):** Sistema que traduce nombres de dominio legibles (como cloud.hexa38.com) a direcciones IP numéricas. Funciona como una agenda telefónica de internet.
- **HTTPS/SSL/TLS:** Protocolos de comunicación segura que cifran los datos entre el navegador y el servidor. El candado verde en el navegador indica que la conexión usa HTTPS.
- **Puerto:** Punto de entrada/salida de datos en un dispositivo. Cada servicio usa un puerto específico: el puerto 80 para HTTP, 443 para HTTPS, 22 para SSH.
- **Firewall (UFW):** Barrera de seguridad que filtra el tráfico de red, permitiendo solo conexiones autorizadas. UFW (Uncomplicated Firewall) es la herramienta de firewall de Ubuntu.
- **SSH (Secure Shell):** Protocolo que permite acceder remotamente a un servidor de forma segura y cifrada, como si estuvieras sentado frente a él.
- **Tunnel (Túnel):** Conexión segura y cifrada que atraviesa internet sin necesidad de abrir puertos en el router. Cloudflare Tunnel conecta el servidor con internet de forma segura.
- **HSTS:** Política de seguridad HTTP que obliga al navegador a usar siempre HTTPS, evitando conexiones inseguras.

2.2 Sistemas Operativos y Servidores

- **Linux (Ubuntu Server):** Sistema operativo de código abierto, gratuito y optimizado para servidores. No tiene interfaz gráfica, lo que lo hace más ligero y seguro. Es el sistema más usado en servidores web del mundo.
- **NUC (Next Unit of Computing):** Mini computadora de Intel, del tamaño de una mano, ideal para servidores domésticos por su bajo consumo energético (~10-15 watts) y funcionamiento silencioso.
- **Swap:** Memoria virtual que usa espacio del disco duro como extensión de la RAM. Cuando la RAM se llena, el sistema usa el swap para evitar quedarse sin memoria.
- **ext4:** Sistema de archivos nativo de Linux. Es robusto, soporta archivos grandes y tiene mecanismos de recuperación ante fallos.
- **NTFS:** Sistema de archivos de Windows. Compatible pero no óptimo para Linux.
- **Montaje de discos:** Proceso de hacer accesible un disco al sistema operativo, asignándole un punto de acceso (como /mnt/nextcloud).

2.3 Contenedores y Virtualización

- **Docker:** Plataforma que empaqueta aplicaciones junto con todas sus dependencias en contenedores aislados. Esto garantiza que la aplicación funcione igual en cualquier sistema.

- **Contenedor:** Entorno ligero y aislado que ejecuta una aplicación. A diferencia de una máquina virtual, comparte el kernel del sistema operativo, lo que lo hace más eficiente.
- **Imagen Docker:** Plantilla de solo lectura que contiene el software y configuración necesarios para crear un contenedor. Es como un molde.
- **Docker Compose:** Herramienta que permite definir y ejecutar múltiples contenedores con un solo archivo YAML (docker-compose.yml).
- **Volumen:** Almacenamiento persistente para contenedores. Los datos en un volumen sobreviven aunque el contenedor se elimine.
- **Red Docker (bridge):** Red virtual que conecta contenedores entre sí, permitiéndoles comunicarse de forma aislada del resto de la red.

2.4 Base de Datos

- **MariaDB:** Sistema de gestión de bases de datos relacional, derivado de MySQL. Almacena toda la información estructurada de Nextcloud: usuarios, archivos, configuración.
- **SQL (Structured Query Language):** Lenguaje estándar para consultar y manipular datos en bases de datos relacionales.
- **Dump:** Exportación completa de una base de datos a un archivo SQL, utilizado para respaldos y migraciones.

2.5 Cache y Rendimiento

- **Redis:** Almacén de datos en memoria (RAM) que funciona como cache. Acelera Nextcloud almacenando temporalmente datos de acceso frecuente.
- **Cache:** Almacenamiento temporal de datos frecuentemente consultados para acelerar el acceso. Reduce la carga en la base de datos.
- **APCu:** Cache local de PHP en memoria, utilizado por Nextcloud para almacenar datos de sesión.

2.6 Seguridad

- **2FA/TOTP:** Autenticación de dos factores. Además de la contraseña, requiere un código temporal generado por una app en el celular (como Google Authenticator). TOTP = Time-based One-Time Password.
- **WHOIS:** Registro público que contiene información sobre quién registró un dominio. Cloudflare oculta estos datos por privacidad.
- **Contraseña de aplicación:** Credencial específica generada para que apps externas accedan a un servicio (como Gmail SMTP) sin usar la contraseña principal.
- **Fuerza bruta:** Ataque que prueba sistemáticamente todas las combinaciones posibles de contraseñas hasta encontrar la correcta.

2.7 Dominio y DNS

- **Dominio:** Nombre único en internet que identifica un sitio web (ej: hexa38.com). Se registra anualmente.
- **Subdominio:** Extensión de un dominio principal (ej: cloud.hexa38.com). Permite organizar diferentes servicios bajo un mismo dominio.

- **Cloudflare:** Servicio que proporciona DNS, protección contra ataques, certificados SSL gratuitos y túneles seguros.
- **Nameservers:** Servidores que resuelven nombres de dominio a direcciones IP. Son la primera parada cuando escribes una URL.

2.8 Automatización

- **Cron/Crontab:** Programador de tareas automáticas en Linux. Permite ejecutar scripts en horarios definidos (ej: backup diario a las 3 AM).
- **Script:** Archivo de texto con instrucciones que se ejecutan secuencialmente para automatizar tareas.
- **Backup:** Copia de seguridad de datos que permite restaurar la información en caso de fallo o pérdida.
- **IaC (Infrastructure as Code):** Práctica de gestionar y provisionar infraestructura tecnológica mediante archivos de código versionables y reproducibles.

3. Disciplinas y Ramas Aplicadas

Este proyecto integra múltiples disciplinas de la computación e ingeniería de sistemas:

Disciplina	Descripción	Aplicación en el Proyecto
Administración de Sistemas (SysAdmin)	Gestión de servidores y sistemas operativos	Instalación de Ubuntu Server, configuración de discos, swap, servicios
Redes de Computadoras	Configuración de conectividad y comunicaciones	DNS, Cloudflare Tunnel, firewall UFW, puertos, SSH
DevOps	Integración de desarrollo y operaciones	Docker, Docker Compose, contenedores, despliegue automatizado
Seguridad Informática	Protección de sistemas y datos	HTTPS, 2FA, firewall, HSTS, actualizaciones automáticas
Bases de Datos	Diseño, gestión y respaldo de datos	MariaDB, dumps SQL, migraciones, restauraciones
Programación / Scripting	Automatización mediante código	Scripts Bash y PowerShell para backups
Cloud Computing	Servicios en la nube y acceso remoto	Cloudflare, túneles, dominio, DNS
Infrastructure as Code	Infraestructura como código reproducible	docker-compose.yml, .env, scripts de configuración

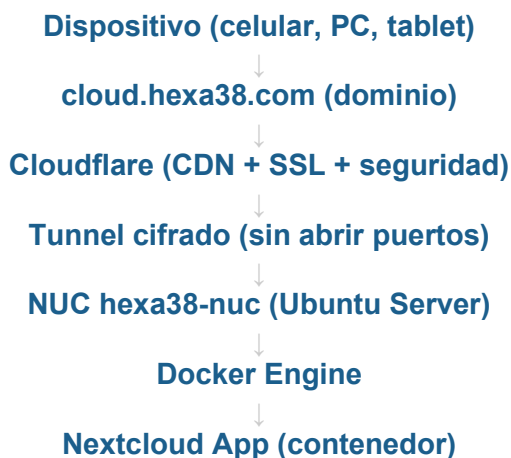
4. Arquitectura del Proyecto

4.1 Componentes físicos

Componente	Especificación	Función
NUC (Intel)	4 GB RAM, WiFi/Ethernet	Servidor físico 24/7
Disco M.2 SSD	297 GB	Sistema operativo Ubuntu Server
Disco externo WD	2 TB (1.8 TB útiles)	Almacenamiento de datos Nextcloud
Router	Conexión a internet	Conectividad de red

4.2 Flujo de datos

El siguiente diagrama muestra cómo fluyen los datos desde cualquier dispositivo hasta el servidor:



4.3 Contenedores Docker

Contenedor	Imagen	Función	Puerto
nextcloud-app	nextcloud:latest	Servidor web Nextcloud	9000:80
nextcloud-db	mariadb:10.11	Base de datos	3306 (interno)
nextcloud-redis	redis:alpine	Cache y sesiones	6379 (interno)
nextcloud-cron	nextcloud:latest	Tareas programadas	N/A
nextcloud-tunnel	cloudflare/cloudflared	Acceso remoto seguro	N/A

5. Desarrollo Paso a Paso

Fase 1: Configuración inicial en PC Windows (Prototipo)

El proyecto comenzó como prototipo en una PC con Windows y Docker Desktop, lo que permitió validar la configuración antes de migrar a un servidor dedicado.

Pasos realizados:

- **Instalación de Docker Desktop:** Plataforma que permite ejecutar contenedores en Windows mediante WSL2 (Windows Subsystem for Linux).
- **Creación del docker-compose.yml:** Archivo que define los 5 servicios (Nextcloud, MariaDB, Redis, Cron, Tunnel) y cómo se relacionan entre sí.
- **Configuración de variables de entorno (.env):** Archivo separado con contraseñas y tokens sensibles, evitando exponerlos en el código.
- **Primer arranque y verificación:** Ejecución de `docker compose up -d` y verificación del funcionamiento con `curl http://localhost:9000/status.php`.

Fase 2: Registro de dominio y acceso remoto

Para acceder a la nube desde cualquier lugar, se necesita un dominio y un mecanismo de conexión segura.

Pasos realizados:

- **Registro de hexa38.com en Cloudflare:** Dominio registrado a precio de costo (\$10.11 USD/año). Cloudflare incluye protección de privacidad WHOIS gratuita.
- **Configuración de Cloudflare Tunnel:** Se creó un túnel seguro que conecta el servidor con Cloudflare sin necesidad de abrir puertos en el router.
- **Asignación del subdominio:** Se configuró `cloud.hexa38.com` como punto de acceso público, apuntando al contenedor Nextcloud (`app:80`).
- **HTTPS automático:** Cloudflare proporciona certificado SSL gratuito. Se configuró HSTS para forzar siempre conexiones cifradas.

Fase 3: Optimización y seguridad

Se realizaron múltiples configuraciones para optimizar el rendimiento y reforzar la seguridad.

Configuraciones aplicadas:

- Dominios de confianza (`trusted_domains`): `cloud.hexa38.com`, `localhost`, IP local
- Redis como cache distribuido y para bloqueo de archivos
- SMTP con Gmail para notificaciones por correo electrónico
- Autenticación de dos factores (2FA/TOTP) para el administrador
- Gestión de usuarios: creación, deshabilitación y asignación de roles
- Resolución de todos los avisos de seguridad del panel de administración
- Migraciones de tipos MIME e índices de base de datos

Fase 4: Sistema de backups automáticos

Se implementó un sistema de respaldo automático que se ejecuta diariamente.

Componentes del backup:

Componente	Método	Contenido
Base de datos	mariadb-dump	Todas las tablas de Nextcloud
Datos de usuarios	docker cp	Archivos, fotos, documentos
Configuración	docker cp	config.php y archivos de config
Archivos Docker	cp	docker-compose.yml, .env, hsts.conf

El backup se ejecuta automáticamente a las 3:00 AM mediante crontab, retiene los últimos 7 respaldos y activa el modo mantenimiento durante el proceso para garantizar la integridad de los datos.

Fase 5: Migración a servidor dedicado (NUC)

La migración del prototipo en PC al servidor dedicado NUC fue el paso crítico del proyecto.

Proceso de migración:

- Instalación de Ubuntu Server 24.04 LTS con kernel HWE en el disco M.2 del NUC
- Formateo del disco externo de 2 TB de NTFS a ext4 y montaje permanente en /mnt/nextcloud
- Instalación de Docker Engine directamente (sin Docker Desktop, ahorrando ~2 GB de RAM)
- Creación de archivos de configuración (docker-compose.yml, .env, hsts.conf)
- Transferencia del backup desde la PC por red local mediante SCP (Secure Copy Protocol)
- Restauración de la base de datos, datos de usuarios y configuración
- Verificación del funcionamiento y redirección del Cloudflare Tunnel al NUC

Fase 6: Hardening del servidor

El hardening (endurecimiento) es el proceso de asegurar un servidor reduciendo su superficie de ataque.

Medidas implementadas:

Medida	Herramienta	Descripción
Swap de 4 GB	fallocate + mkswap	Extiende la RAM virtual para evitar que el sistema se quede sin memoria
Firewall	UFW	Solo permite SSH (puerto 22) y Nextcloud (puerto 9000)
Actualizaciones automáticas	unattended-upgrades	Instala parches de seguridad automáticamente
Rotación de logs	Docker daemon.json	Limita logs a 10 MB por contenedor (máximo 3 archivos)

Proxy de confianza	trusted_proxies	Identifica correctamente la IP real del usuario a través de Cloudflare
Backup diario	Crontab + script Bash	Respaldo automático a las 3:00 AM con retención de 7 días

Fase 7: Resolución de avisos y acceso remoto SSH

Tras la migración, el panel de administración de Nextcloud mostró avisos de seguridad que fueron resueltos uno a uno hasta obtener el estado «Ha pasado todos los controles».

Avisos resueltos:

Aviso	Solución aplicada
Directorio de datos accesible desde internet	Configuración de Apache para denegar acceso directo al directorio /data mediante data-protect.conf
Errores en los registros	Limpieza del archivo de log (errores históricos de la migración)
2FA no obligatorio	Activación de la obligatoriedad de autenticación de dos factores para todos los usuarios
Migraciones MIME pendientes	Ejecución de <code>occ maintenance:repair --include-expensive</code>
Índices de base de datos faltantes	Ejecución de <code>occ db:add-missing-indices</code>
Encabezado HSTS no configurado	Creación de <code>hsts.conf</code> montado como volumen en Apache
Región telefónica no configurada	Configuración de <code>default_phone_region</code> a PE (Perú)
Ventana de mantenimiento no configurada	Configuración de <code>maintenance_window_start</code> a las 5 AM UTC

Instalación de Tailscale (SSH remoto seguro):

Para administrar el servidor desde cualquier lugar del mundo, se instaló Tailscale, una VPN mesh que crea una red privada entre dispositivos sin necesidad de abrir puertos ni configurar routers.

- **Tailscale:** Servicio de red privada virtual (VPN) basado en WireGuard. Cada dispositivo recibe una IP privada (100.x.x.x) accesible desde cualquier lugar.
- **Tailscale SSH:** Permite conexiones SSH autenticadas por Tailscale, eliminando la necesidad de gestionar llaves SSH manualmente.
- **WireGuard:** Protocolo VPN moderno, rápido y seguro que Tailscale usa internamente para cifrar el tráfico.

Con Tailscale configurado, el administrador puede conectarse al servidor desde cualquier red (casa, oficina, celular) ejecutando simplemente: `ssh aldo@hexa38-nuc` o `ssh aldo@100.91.119.52`.

Acceso remoto final:

Servicio	URL / Comando	Tecnología
Nextcloud (web)	https://cloud.hexa38.com	Cloudflare Tunnel

SSH al servidor	ssh aldo@hexa38-nuc	Tailscale
SSH local	ssh aldo@192.168.1.69	Red local WiFi/Ethernet

6. Archivos de Configuración Clave

6.1 docker-compose.yml

Este archivo define toda la infraestructura como código. Cada sección declara un servicio (contenedor) con su imagen, configuración y relaciones.

```
services:
  db:
    image: mariadb:10.11          # Software a usar
    restart: always              # Reinicio automático si falla
    volumes:
      - /mnt/nextcloud/db:/var/lib/mysql # Datos persistentes
    environment:
      - MYSQL_PASSWORD=${MYSQL_PASSWORD} # Variables desde .env

  redis:
    image: redis:alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}

  app:
    image: nextcloud:latest
    ports:
      - 9000:80                  # Puerto externo:interno
    depends_on:                  # Orden de inicio
      - db
      - redis
    volumes:
      - /mnt/nextcloud/html:/var/www/html
      - /mnt/nextcloud/data:/var/www/html/data

  cron:
    image: nextcloud:latest
    entrypoint: /cron.sh          # Ejecuta tareas programadas

  tunnel:
    image: cloudflare/cloudflared:latest
    command: tunnel run --token ${CLOUDFLARE_TOKEN}
```

Explicación de directivas:

Directiva	Significado
image	Imagen Docker a usar (software y versión)
restart: always	Si el contenedor falla, Docker lo reinicia automáticamente
volumes	Conecta carpetas del disco con el contenedor (persistencia de datos)
environment	Variables de configuración (contraseñas, nombres de host)
ports	Mapeo de puertos: puerto_externo:puerto_interno
depends_on	Define el orden de inicio de los contenedores
command	Comando que ejecuta el contenedor al iniciar
entrypoint	Punto de entrada principal del contenedor

networks	Red virtual para comunicación entre contenedores
----------	--

6.2 Script de backup (backup.sh)

El script de backup sigue una lógica secuencial para garantizar la integridad de los datos:

Paso	Acción	Propósito
1	Activar modo mantenimiento	Evitar cambios durante el backup
2	Exportar base de datos (dump)	Respaldo de toda la información estructurada
3	Copiar datos de usuarios	Respaldo de archivos, fotos, documentos
4	Copiar configuración	Respaldo de config.php y ajustes
5	Copiar archivos Docker	Respaldo de docker-compose.yml y .env
6	Desactivar modo mantenimiento	Reanudar el servicio normal
7	Limpiar backups antiguos	Mantener solo los últimos 7 respaldos

7. Infrastructure as Code (IaC)

7.1 ¿Qué es Infrastructure as Code?

Infrastructure as Code (Infraestructura como Código) es una práctica moderna que consiste en gestionar y provisionar infraestructura tecnológica mediante archivos de código, en lugar de configuraciones manuales. Esto permite que la infraestructura sea versionable, reproducible y documentada automáticamente.

7.2 Herramientas principales

Herramienta	Función	Uso en este proyecto
Docker Compose	Define servicios como código	Ya implementado: docker-compose.yml define toda la infraestructura
Terraform	Provisiona recursos en la nube	Potencial: crear VPS en Oracle Cloud o AWS automáticamente
Ansible	Configura servidores automáticamente	Potencial: automatizar instalación de Docker y configuración
Cloud-init	Inicializa servidores al arrancar	Potencial: configuración automática de Ubuntu Server
Git	Control de versiones	Potencial: versionar todos los archivos de configuración

7.3 Beneficios de IaC

- **Reproducibilidad:** La misma configuración produce el mismo resultado cada vez. Si el servidor falla, se puede reconstruir desde cero en minutos.
- **Versionado:** Los cambios en la infraestructura quedan registrados en Git, permitiéndote ver qué cambió, cuándo y por qué.
- **Documentación automática:** El código mismo es la documentación. No hay pasos manuales que olvidar.
- **Escalabilidad:** Desplegar el mismo servicio en múltiples servidores es tan fácil como copiar los archivos.

7.4 Cómo reproducir este proyecto con IaC

Con los archivos de este proyecto (docker-compose.yml, .env, hsts.conf, backup.sh), cualquier persona puede replicar la nube personal completa siguiendo estos pasos:

```
# 1. Instalar Docker
curl -fsSL https://get.docker.com | sudo sh

# 2. Copiar archivos de configuración
scp -r usuario@servidor-origen:~/nextcloud ~/nextcloud

# 3. Levantar toda la infraestructura
cd ~/nextcloud && docker compose up -d
```

```
# 4. Restaurar backup
docker exec -i nextcloud-db bash -c \
'mariadb -u root -p"$MYSQL_ROOT_PASSWORD" nextcloud' < backup/database.sql
```

Con estos 4 pasos, toda la infraestructura queda funcionando: Nextcloud, base de datos, cache, tareas programadas y acceso remoto. Esto es el poder de Infrastructure as Code.

8. Costos del Proyecto

Concepto	Costo	Frecuencia
Dominio hexa38.com (Cloudflare)	\$10.11 USD	Anual
Cloudflare Tunnel	Gratis	-
Nextcloud (código abierto)	Gratis	-
Docker Engine	Gratis	-
Ubuntu Server	Gratis	-
NUC (hardware)	Ya existente	-
Disco WD 2 TB	Ya existente	-
Electricidad del NUC (~15W)	~\$2-3 USD	Mensual
TOTAL ANUAL	~\$46 USD	

Comparación de costos anuales:

Servicio	Almacenamiento	Costo anual
Google One	2 TB	\$120 USD
Dropbox Plus	2 TB	\$144 USD
iCloud+	2 TB	\$131 USD
Nube Personal (este proyecto)	1.7 TB	\$46 USD

El proyecto representa un ahorro del 60-68% respecto a servicios comerciales, con la ventaja adicional de privacidad total, usuarios ilimitados y control completo sobre los datos.

9. Resultados

El proyecto logró todos los objetivos planteados:

Objetivo	Estado	Detalle
Nube personal funcionando	Completado	Nextcloud 33.0.2 operativo 24/7
Acceso remoto	Completado	cloud.hexa38.com accesible desde cualquier lugar
Almacenamiento	Completado	1.7 TB disponibles en disco de 2 TB
Seguridad	Completado	HTTPS, 2FA, firewall, actualizaciones automáticas
Backups	Completado	Respaldo diario automático a las 3 AM
Bajo costo	Completado	~\$46 USD/año vs \$120+ en servicios comerciales
Privacidad	Completado	Control total sobre los datos, sin terceros
Multi-dispositivo	Completado	Apps para Android, iOS, Windows, Mac, Linux

Estadísticas del servidor

Métrica	Valor
RAM total	3.7 GB + 7.7 GB swap
RAM usada (sistema + Docker)	~985 MB
Disco disponible	1.7 TB de 1.8 TB
Contenedores activos	5
Tiempo de arranque	< 2 minutos
Conexiones del tunnel	Santiago (SCL) + Lima (LIM)

10. Conclusiones

- Este proyecto demuestra que es posible integrar múltiples disciplinas de la computación (redes, sistemas, seguridad, bases de datos, DevOps) en una solución práctica y funcional.
- Los contenedores Docker simplifican enormemente el despliegue de servicios complejos. Lo que antes requería días de configuración manual, ahora se define en un archivo YAML y se levanta con un solo comando.
- La automatización (backups, actualizaciones, cron) reduce errores humanos y garantiza consistencia. Un servidor bien automatizado requiere mínima intervención manual.
- El self-hosting (alojamiento propio) es una alternativa viable, económica y educativa a los servicios en la nube comerciales, especialmente para usuarios que valoran la privacidad y el control de sus datos.
- Infrastructure as Code (IaC) permite reproducir toda la infraestructura de forma consistente, facilitando migraciones, recuperación ante desastres y escalamiento a nuevos proyectos.
- El proyecto evolucionó desde un prototipo en PC hasta un servidor dedicado 24/7, demostrando un ciclo completo de desarrollo: prototipado, pruebas, optimización, migración y hardening.

11. Referencias y Recursos

Tecnologías utilizadas

Tecnología	Sitio web	Descripción
Nextcloud	https://nextcloud.com	Plataforma de nube personal de código abierto
Docker	https://docker.com	Plataforma de contenedores
Cloudflare	https://cloudflare.com	DNS, seguridad y túneles
Ubuntu Server	https://ubuntu.com/server	Sistema operativo para servidores
MariaDB	https://mariadb.org	Sistema de gestión de bases de datos
Redis	https://redis.io	Almacén de datos en memoria

Herramientas de IaC (referencia)

Herramienta	Sitio web	Descripción
Terraform	https://terraform.io	Provisionamiento de infraestructura en la nube
Ansible	https://ansible.com	Automatización de configuración de servidores
Git	https://git-scm.com	Control de versiones de código